

Analysing the algorithm in terms of efficiency

- ① Time complexity
- ② Space complexity

* Problem - Algorithm *

Related to lower bound theory

→ lower bounds is defined for problems not algorithms

→ Feasible / infeasible, tractable / intractable, easy / hard, problem / algorithm

* This terminology is related to "NP-completeness" concept

* The type of the computer on which an algorithm executes is irrelevant. It does not change the nature of the algorithm

* Selecting an appropriate input parameter ⇒ Sorting, searching, finding max element in a list
Multiplying two matrices → Input parameter: $n \times m$

Primality testing ⇒ Input parameter is the number of digits in decimal / binary representation of n

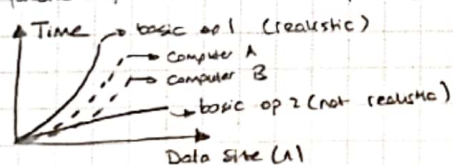
Spell Checking Problem ⇒ number of characters in the text

Graph Traversal Problem ⇒ Input parameter: $f(n, m)$
number of vertices
number of edges

Basic Operation

Usually the most important operation OR an operation that the algorithm performs repeatedly OR the operation contributing the most to the total execution time

* A basic operation is called realistic if the actual execution time is bounded by a linear function of the derived complexity.



- Searching a list
 - Sorting a list
 - Finding Max/Min
- } Basic operation is comparisons

- Multiplying two matrices } Basic operation is multiplication

- Traversing a tree / graph } Basic operation is accessing a node

- Spell checking } Basic operation is number of dictionary lookups a higher level operation it characterizes better

$f(n) = \dots$ The result does not only depend on the data size. It also depends on the characteristics of the input.

Problem

Sorting

$\Omega(n \log n)$

Algorithm

Quick sort

$O(n \log n)$

} There is no way to try to develop a better algorithm for sorting

There are problems whose lower bounds are very high

Ex: Tower of Hanoi
 $2^B - 1$

Searching: 1, 2, ..., n. The algorithm will not take the same time for all data of size n. If the searched element appears at the beginning the algo will end immediately.

T_n = The set of all inputs of size n.

$T(I)$ = The number of basic operations performed by the algorithm for input I.

③ Best case complexity of an algorithm: $B(n) = \min \{T(I) : I \in T_n\}$

④ Worst case " " " " " " : $W(n) = \max \{T(I) : I \in T_n\}$

⑤ Average case " " " " " " : $A(n) = \sum_{I \in T_n} T(I) \cdot P(I)$

Probability of occurrence of input I

$$T(I_1) \cdot P(I_1) + T(I_2) \cdot P(I_2) + \dots + W(n)$$

$$A(n) = \sum_{i=1}^{W(n)} i \cdot P_i \quad \rightarrow \text{probability that the algorithm performs } i \text{ basic op.}$$

Ex) An algorithm for searching in a list of size n.

Suppose that $n = 10 \rightarrow \infty$ possible inputs exist.

Suppose that $n = 10$ and the elements in the list

are distinct and we know these 10 particular elements.

We divide the elements into 10 classes such that each class contains the inputs for which the searched element appears at the same position.

Only 10 possible inputs exist (we regard classes as inputs).

$$A(n) = T(I_1) \cdot P(I_1) + \dots + T(I_{10}) \cdot P(I_{10})$$

10! possible inputs exist

Even under this restriction there exist a huge number of possible inputs.

$B(n) \Rightarrow$ lower limit

$W(n) \Rightarrow$ upper limit

$A(n)$

for instance, if $A(n)$ is good but $W(n)$ is very bad, then I might not want to use this algorithm.

Calculating $B(n)$ and $W(n)$ is usual easier than calculating $A(n)$.

Calculating $A(n)$

We need to determine suitable values for the probabilities (i.e. a probability dist. for the input). We need to know and apply the elements of probability theory. Formula for $A(n)$ (usually involves summation or recurrence relation).

$$\text{Ex) } A(n) = \sum_{i=1}^n \left(\frac{i^2}{5} + \log i \right)$$

We should convert the formula into "closed form" $A(n) = \frac{n^3}{10}$

General Framework for the Analysis of an Algorithm

→ Decide on a parameter(s) indicating the input size

→ Identify the algorithm's basic operation

→ Count how many times the basic operation is performed for an input of size n.

(The most difficult step in this framework)

For some algorithms, complexity does not depend on the nature of the data. If this is the case $A(n) = W(n) = B(n)$

In fact complexity analysis means counting.

If the result is in the form of a summation or recurrence relation then convert it to a closed form expression by some mathematical manipulation (using app. is also sometimes permissible) $A(n) \approx$

Usually iteration algorithms result in summations and recursive algorithms result in recurrences.

$$A(n) = \sim$$

$A(n) \in \mathcal{O}(\sim)$ = approximation

$$A_1(n) = 3n^2 + \frac{\log \log n}{\sqrt{n}}$$

$$A_2(n) = \frac{10 + 2(n - 5n)}{n^{3/2}}$$

Which one is better?

Principles

→ Look at the growth rate of the complexity function as $n \rightarrow \infty$

Analyzing the behavior of an algorithm as the input size tends to infinity is called asymptotic analysis

→ Look at the growth rate of the complexity function instead of the actual values it takes for different n 's (data sizes).

The rate of increase in the complexity function of an algorithm as the input size increases is called the growth rate.

Ex) $f_1(n) = \frac{n^2}{2} - \frac{n}{2}$

$$f_2(n) = n^2$$

n	$\frac{n^2}{2}$	$\frac{n^2}{2} - \frac{n}{2}$	n^2
10	50	45	100
1000	500000	499500	1000000
10000	-	-	-
100000	-	-	-

these 2 functions have the same growth rate

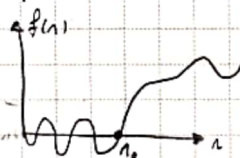
• Eliminate low-order terms

$f_1(n)$ and $f_2(n)$ are approximately equivalent

• Eliminate constant factors

The complexity $f(n)$ of the algorithm is of order $g(n)$

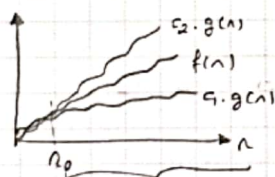
Let F be the set of all functions $f(n): \mathbb{N} \rightarrow \mathbb{R}$ that are eventually positive.



is a number n_0 $f(n) > 0$ $\forall n > n_0$

Let $f(n)$ and $g(n) \in F$. $f(n) \in \mathcal{O}(g(n))$ iff there exists positive constant c_1, c_2 and n_0 s.t.

$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ whenever $n > n_0$



$f(n)$ grows at the same rate as $g(n)$

intuitive, informal meaning of notation.

$\mathcal{O}(g(n))$ represents a lot of functions.

$f(n) \in \mathcal{O}(g(n))$

function set of functions

$$\mathcal{O}(n^2) = \{n^2, 5n^2, 3n^2, n^2 + \log n, \dots\}$$

$$\mathcal{O}(n^2) = \mathcal{O}(5n^2) = \mathcal{O}(3n^2)$$

$f(n) \in \mathcal{O}(n^2) \rightarrow$ conventional

Ex) $n^2 \in \mathcal{O}(5n^2 + n)$. If we can show that constants c_1, c_2 and n_0

exist s.t. the following is true, then we are done:

$$c_1 \cdot (5n^2 + n) \leq n^2 \leq c_2 \cdot (5n^2 + n) \text{ for } n \geq n_0$$

$$\begin{aligned} c_1 &= \frac{1}{10} \\ c_2 &= 1 \\ n_0 &= 1 \end{aligned} \quad \frac{5n^2 + n}{10} \leq n^2 \leq 5n^2 + n \text{ for } n \geq 1$$

Ex] $n^2 \notin \Theta(n^3)$ We need to prove that it is impossible to find such c_1, c_2 and n_0 assume the opposite.

$$c_1 n^2 \leq n^2 \leq c_2 n^3 \text{ for } n \geq n_0$$

$c_1 n \leq 1 \leq c_2 n$ as $n \rightarrow \infty, c_1 \rightarrow 0$ but c_1 must be a positive constant

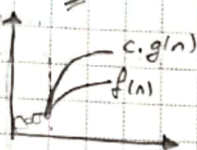
Θ determines an equivalence relation on the set of functions (an equivalence relation on F)

→ Reflexivity: $f(n) \in \Theta(f(n))$

→ Symmetry: $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$

→ Transitivity: $f(n) \in \Theta(g(n))$ and $g(n) \in \Theta(h(n)) \Rightarrow f(n) \in \Theta(h(n))$

Big-Oh (O) Upper Bound: A function $f(n) \in O(g(n))$ iff there exists positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ whenever $n \geq n_0$.



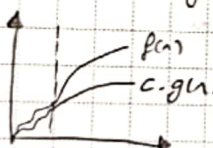
* $f(n)$ grows at the same rate or slower than $g(n)$

* O (Big-Oh) notation gives us an upper bound for complexity.

Ex] $5n^2 \in O(n^2)$

$5n^2 \leq cn^2$ for some $n \geq n_0$ $\left[\begin{matrix} c=10 \\ n_0=1 \end{matrix} \right]$. Finding one such pair of c and n_0 is sufficient.

Omega (Ω) Lower Bound: A function $f(n) \in \Omega(g(n))$ iff there exists positive constants c and n_0 such that $c \cdot g(n) \leq f(n)$ whenever $n \geq n_0$.



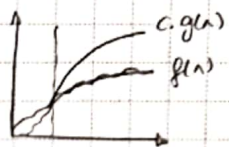
* $f(n)$ grows at the same rate or faster than $g(n)$

* Ω notation gives a lower bound for complexity.

Matrix multiplication problem $\Rightarrow$ Classical way $O(n^3) \rightarrow$ A better algo $- O(n^{2.71})$
 $\hookrightarrow$ Lower bound: $\Omega(n^2)$

Ex] $5n^2 \in \Omega(n^2)$ $\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$
 $n^3 \in \Omega(n^2)$

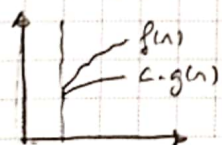
Little-Oh (o): A function $f(n) \in o(g(n))$ iff for every positive constant c , there is a positive integer n_0 such that $f(n) < c \cdot g(n)$ whenever $n \geq n_0$.



* $f(n)$ grows slower than $g(n)$

Ex] $n^2 \in o(n^3)$ $n^2 \in O(n^2)$
 $n^2 \notin \Omega(n^2)$

Little-Omega (ω): A function $f(n) \in \omega(g(n))$ iff for every positive constant c there is a positive integer n_0 such that $c \cdot g(n) < f(n)$ whenever $n \geq n_0$.



* $f(n)$ grows faster than $g(n)$

Ex] $n^3 \in \omega(n^2)$
 $n^3 \notin \omega(n^3)$

Asymptotic Notation - Relational Operations

$\Theta \rightarrow =$ $\Omega \rightarrow \geq$ $\omega \rightarrow >$ + the most frequently used operator is O . Then Θ , then Ω .
 $O \rightarrow \leq$ $o \rightarrow <$

$f(n, m) \in O(g(n, m))$ iff there exists positive constants c and n_0 , such that $f(n, m) \leq c \cdot g(n, m)$ whenever $n, m \geq n_0$.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{cases} 0; f(n) \in o(g(n)) \\ c; c > 0; f(n) \in \Theta(g(n)) \\ \infty; f(n) \in \omega(g(n)) \\ 1; f(n) \in \sim(g(n)) \end{cases}$$

special case tilda

L'Hospital's Rule: Let $f(x)$ and $g(x)$ be functions that are differentiable for sufficiently large real numbers x .

If $\lim_{x \rightarrow \infty} f(x) = \infty$ and $\lim_{x \rightarrow \infty} g(x) = \infty$ or If $\lim_{x \rightarrow \infty} f(x) = 0$ and $\lim_{x \rightarrow \infty} g(x) = 0$; then $\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{f'(x)}{g'(x)}$.

Properties let $f(n), g(n) \in F \rightarrow$ set of functions

- * $f(n) \in O(g(n)) \Leftrightarrow g(n) \Omega f(n)$
- * $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$
- * $f(n) \in \omega(g(n)) \Leftrightarrow f(n) \in O(g(n))$ and $f(n) \in \Omega(g(n))$
- * $f(n) \in o(g(n)) \Leftrightarrow g(n) \in \omega(f(n))$
- * $f(n) \in O(g(n)) \Rightarrow f(n) \in O(g(n))$
- * $f(n) \in \omega(g(n)) \Rightarrow f(n) \in \Omega(g(n))$
- * Given $c > 0; \Theta(f(n)) = \Theta(c \cdot f(n))$ Also true for other symbols except $\sim$
- * $f(n) \in O(g(n)) \Leftrightarrow O(f(n)) \subseteq O(g(n))$
 $n \leftarrow O(n^2) \subseteq O(n^3)$
- * $f(n) \in O(g(n)) \Leftrightarrow O(f(n)) \subseteq O(g(n))$
- * $O(f(n)) = O(g(n)) \Leftrightarrow \Theta(f(n)) = \Theta(g(n)) \Leftrightarrow \Omega(f(n)) = \Omega(g(n))$
- * $f(n) \in \sim(g(n)) \Rightarrow f(n) \in \Theta(g(n))$

* Given two functions $f(n), g(n) \in F$
 $f(n)$ and $g(n)$ have the same order if $f(n) \in \Theta(g(n))$
 $f(n)$ has smaller order than $g(n)$ if $O(f(n)) \subset O(g(n))$

* $f(n)$ and $g(n)$ are not comparable if $f(n) \notin O(g(n))$ and $g(n) \notin O(f(n))$

Ex Functions involving modulo operations

Popularly used classes:

- $\rightarrow O(1)$ constant order (usually in best case complexity) (fastest)
- $\rightarrow O(\log \log n), O(\log \log n^{1/2})$ (very fast)
- $\rightarrow O(\log n)$ logarithmic order
- * The base of the logarithm does not matter for complexity classes. $\log_a n = \log_b n \cdot \log_b a$

$\rightarrow O(n^a); 0 < a < 1$ the most common of this class is $O(\sqrt{n})$

for such a , n^a grows faster than $\log n$

$$\lim_{n \rightarrow \infty} \frac{n^a}{\log n} = \lim_{n \rightarrow \infty} \frac{a \cdot n^{a-1}}{1/n} = \lim_{n \rightarrow \infty} \frac{a}{1-a} \cdot \frac{1}{n} = \lim_{n \rightarrow \infty} \frac{a}{n^a} = \lim_{n \rightarrow \infty} a \cdot n^a = \infty$$

* n where $0 < a < 1$ grows faster than $\log n$ $n^a \in (\log n)$, where $0 < a < 1$

$\rightarrow O(n)$ (linear order, used frequently)

$\rightarrow O(n \log n)$, (typical in divide and conquer algorithms) (e.g. quick sort, merge sort)

$\rightarrow O(n^2)$, quadratic order

$\rightarrow O(n^3)$, cubic order

$\rightarrow O(n^a)$ for $a > 1$ exponential order a^n grows faster than n^a , $a^{\sqrt{n}}$, $a^{\log n}$ are exponential

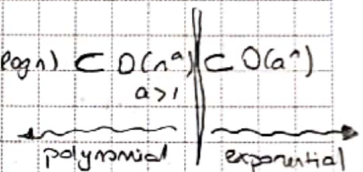
$\rightarrow O(n \cdot a^n); a > 1$

$\rightarrow O(n!)$, factorial order very very slow algorithms

$\rightarrow O(a^{n^b})$ If an algorithm generates all permutations of a list it will have factorial complexity

$$O(1) \subset O(\log \log n) \subset O(\log n) \subset O(n^a) \subset O(n) \subset O(n \cdot \log n) \subset O(n^a) \subset O(a^n) \subset O(n \cdot a^n) \subset O(a^{n^b})$$

for $0 < a < 1$



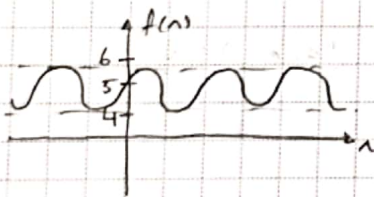
* an algorithm whose complexity is at most $O(f(n))$, where $f(n)$ is a polynomial of n , is called a polynomial-time algorithm

* an algorithm whose complexity is at least $\Omega(a^n)$, where $a > 1$ is called an exponential time algorithm

* This distinction is the most important thing in the running time of the algorithm

* Polynomial-time algorithms are feasible (that is solvable by a computer in reasonable time) whereas exponential are infeasible

Ex) $f(n) = 5 + \sin(n)$ what is this complexity



$f(n) \in O(1)$ constant complexity $\Rightarrow$ Bounded above by a constant

$$f(n) \leq c \cdot 1 \text{ for } n \geq n_0$$

$$\Downarrow$$

$$n_0 = 1$$

$f(n) \geq c \cdot 1$ for $n \geq n_0 \Rightarrow$ Bounded below by a constant

$$\Downarrow$$

$$n_0 = 1$$

$f(n) \in \Theta(1)$

Ex) Prove that $\sum_{k=1}^n k^p$ is $\Theta(n^{p+1})$ for $p > 1$ a fixed constant

proof: $\rightarrow$ First, find an upper bound $\sum_{k=1}^n k^p = 1^p + 2^p + 3^p + \dots + n^p \leq n \cdot n^p = n^{p+1}$

n terms
each term $\leq n^p$

$$\sum_{k=1}^n k^p \leq n^{p+1} \Rightarrow \in O(n^{p+1}) \text{ for } p > 1$$

→ Find a lower bound

$$\sum_{k=1}^n k^p = \sum_{k=1}^{\lfloor \frac{n}{2} \rfloor} k^p + \sum_{k=\lfloor \frac{n}{2} \rfloor + 1}^n k^p \geq \sum_{k=\lfloor \frac{n}{2} \rfloor + 1}^n k^p$$

floor: $\lfloor x \rfloor$ on integer $\leq x$

ceiling: $\lceil x \rceil$ on integer $\geq x$

$$= (\lfloor \frac{n}{2} \rfloor + 1)^p + (\lfloor \frac{n}{2} \rfloor + 2)^p + \dots + n^p$$

$$\lfloor \frac{n}{2} \rfloor \text{ terms} \geq \frac{n}{2} \text{ terms} \quad \text{each term} \geq (\frac{n}{2})^p \geq \frac{n}{2} (\frac{n}{2})^p = \frac{n^{p+1}}{2^{p+1}} \in \Omega(n^{p+1})$$

Ex] $f(n) = \frac{n(n+1)}{2}$; $g(n) = n^2$ find the asymptotic relationship between these 2 functions.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{n^2 + n}{2n^2} = \frac{1}{2} \quad f(n) \in O(g(n))$$

Ex] Show that if $p(n)$ is a polynomial of degree k , where the coefficient of n^k is greater than 0, then $p(n) \in O(n^k)$

$$p(n) = a_n \cdot n^k + a_{n-1} \cdot n^{k-1} + \dots$$

Ex] Show that any polynomial function has smaller order than any exponential function i.e. $[O(n^k) < O(a^n)]$ for $k > 0, a > 1$

$$\lim_{n \rightarrow \infty} \frac{n^k}{a^n} = \frac{k(n^{k-1})}{a^n \cdot a} = \lim_{n \rightarrow \infty} \frac{k(n^{k-1})}{(a^n)(a) a^n} = \lim_{n \rightarrow \infty} \frac{k!}{(a^n)^k a^n} = 0$$

after applying L'Hospital rule for k times

Ex] Computer orders of growth of $n!$ and 2^n

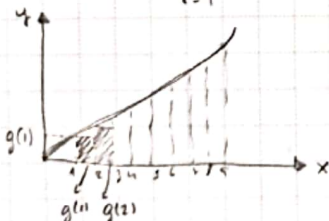
$n! \approx \sqrt{2\pi n} \cdot (\frac{n}{e})^n$ for large $n \Rightarrow$ Stirling's formula $\rightarrow$ when we see factorial

$$\lim_{n \rightarrow \infty} \frac{\sqrt{2\pi n} \cdot (\frac{n}{e})^n}{2^n} = \lim_{n \rightarrow \infty} \underbrace{(2\pi n)}_{\text{constant}} \cdot \left(\frac{n}{2e}\right)^n = \infty$$

④ In analyzing an algorithm, we usually arrive at a summation $\sum$ and then convert this summation into closed form.

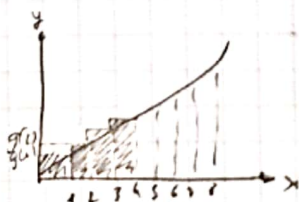
Sometimes this $\sum$ is too complex, we can squeeze the summation between 2 integrals.

Let $f(n) = \sum_{i=1}^n g(i)$, where g is a nondecreasing function.



$$g(1) \cdot 1 + g(2) \cdot 1 + g(3) \cdot 1 + \dots + g(n) \cdot 1 = \sum_{i=1}^n g(i) = f(n)$$

$$\int_1^n g(x) dx \Rightarrow \text{the entire area under the curve} = f(n) \leq \int_1^n g(x) dx$$



$$g(1) \cdot 1 + g(2) \cdot 1 + g(3) \cdot 1 + \dots + g(n) \cdot 1$$

$$\int_0^1 g(x) dx \leq f(n) \quad f(n) \geq \int_0^n g(x) dx$$

Ex) $f(n) = \sum_{i=1}^n i^2$ non-decreasing

$$\int_0^n g(x) dx \leq f(n) \leq \int_1^{n+1} g(x) dx \quad \rightarrow \quad \frac{x^3}{3} \Big|_0^n \leq f(n) \leq \frac{x^3}{3} \Big|_1^{n+1}$$

$$\int_0^n x^2 dx \leq f(n) \leq \int_1^{n+1} x^2 dx \quad \rightarrow \quad \frac{n^3}{3} \leq f(n) \leq \frac{(n+1)^3}{3} - \frac{1}{3} \text{ constant} \Rightarrow f(n) \in \Theta(n^3)$$

prev. method: $\sum_{i=1}^n i^2 = 1^2 + 2^2 + \dots + n^2 \leq n \cdot n^2 = n^3 \Rightarrow O(n^3)$

n terms
each $\leq n^2$

Ex) Sum of logarithms $L(n) = \sum_{i=1}^n \log(i) = \log(n!)$ $\Rightarrow L(n) = \log(1) + \log(2) + \dots + \log(n) \leq n \cdot \log(n)$

n terms
each $\leq \log(n)$

for lower bound: $L(n) = \sum_{i=1}^{n/2} \log(i) + \sum_{i=\lfloor n/2 \rfloor + 1}^n \log(i) \geq \sum_{i=\lfloor n/2 \rfloor + 1}^n \log(i)$

this summation $\geq \frac{n}{2} \cdot \log(\frac{n}{2})$ $L(n) \in \Omega(n \log n)$

there are $n - (\lfloor \frac{n}{2} \rfloor) = \lceil \frac{n}{2} \rceil$
each term $\geq \lfloor \frac{n}{2} \rfloor + 1 \geq \frac{n}{2}$

$$\int_1^n \log(x) dx \leq L(n) \leq \int_1^{n+1} \log(x) dx \quad \rightarrow \quad x \log x - x \Big|_1^n \leq L(n) \leq x \log x - x \Big|_1^{n+1}$$

$$\int_1^n \ln(x) dx \leq L(n) \leq \int_1^{n+1} \ln(x) dx \quad \rightarrow \quad n \ln n - n \leq L(n) \leq (n+1) \ln(n+1) - (n+1) + 1$$

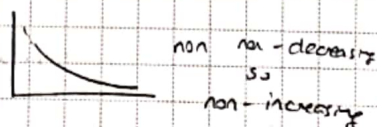
$$\frac{(n+1) \ln(n+1) - n}{O(n \ln(n))} \quad L(n) \in O(n \cdot \log n)$$

$$e^{n \ln(n) - n} \leq e^{L(n)} \leq e^{(n+1) \ln(n+1) - n}$$

$$\frac{e^{n \ln(n) - n}}{e^n} \leq n! \leq \frac{e^{(n+1) \ln(n+1)}}{e^n} \quad a^{n \cdot \log_a n} = n^n$$

Harmonic Series

$$H(n) = \sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$$



$$\int_1^{n+1} g(x) dx \leq H(n) \leq \int_1^n g(x) dx \quad \text{for a non-increasing function.}$$

$$\int_1^{n+1} \frac{1}{x} dx \leq H(n) \leq \int_1^n \frac{1}{x} dx \quad \rightarrow \quad \ln x \Big|_1^{n+1} \leq H(n) \leq \ln x \Big|_1^n \rightarrow \ln(n+1) \leq H(n) \leq \ln(n) - \ln(1)$$

so, this method works for finding Θ , but not for upper bound.

$$H(n) = 1 + \sum_{i=2}^n \frac{1}{i} \quad H(n) \leq 1 + \int_1^n g(x) dx = 1 + \ln(x) \Big|_1^n = 1 + \ln(n) - \ln(1) \therefore H(n) \in O(\log n)$$

$$= \ln(n) + 1$$

$$\ln H(n) \in O(\log n)$$

Linear (Sequential) Search

Pseudo Code:

```

function Linear Search (L[1:n], x)
    for i = 1 to n do
        if (L[i] == x) do
            return(i)
        end if
    end for
end
    
```

A comparison-based searching or sorting algorithm is an algorithm that is based on making comparisons involving list elements and then making decisions based on those comparisons.

Best Case: If $x = L[1]$, then best case occurs $B(n) = 1 \in O(1)$

Worst Case: If $x = L[n]$ or x does not occur in the list $W(n) = n \in O(n)$

Average Case: Assume that the prob. of a successful search is P , where $0 \leq P \leq 1$

Assume that x can equally likely be found in any position

Assume that the elements in the list are distinct. Let P_i = prob. that i operations will be done. (i.e., prob. that x occurs in the i^{th} position, that is, prob. that $x = L[i]$)

$$= \begin{cases} \frac{P}{n} & \text{for } 1 \leq i \leq n-1 \\ \frac{P}{n} & \text{for } i = n \end{cases}$$

Prob. that $x = L[i]$ equals $\frac{P}{n}$ for $1 \leq i \leq n-1$ Prob. that $x = L[i]$ or x does not exist $\frac{P}{n} + (1-P)$ for $i = n$

$$A(n) = \sum_{i=1}^n i \cdot P_i = \left(\sum_{i=1}^{n-1} i \cdot \frac{P}{n} \right) + n \cdot \left[\frac{P}{n} + (1-P) \right] = \frac{P}{n} \sum_{i=1}^{n-1} i + n \left(\frac{P}{n} + 1 - P \right) = \frac{P}{n} \frac{n(n-1)}{2} + p + n(1-p)$$

• average complexity is also linear.

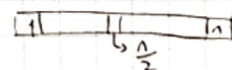
$$= \frac{P(n-1)}{2} + p + n(1-p)$$

$$= \frac{Pn}{2} - \frac{P}{2} + p + n - np$$

$$= n \left(1 - \frac{P}{2} \right) + \frac{P}{2} \in O(n)$$

Binary Search

Assumes that the list is sorted. Otherwise, binary search does not work.



Best Case: $B(n) = 1 \in O(1)$ if x equals the middle element.

Worst Case: It is not easy to analyze the worst-case for general n . So firstly, suppose that $n = 2^k - 1$ for some $k \in \mathbb{Z}^+$

If x is not in the list or $x = L[1]$ or $x = L[n]$ then the worst-case occurs.

Compare x with $L[2^{k-1}]$, $L[2^{k-2}]$, ..., $L[1]$ there are k comparisons.

$$k = \log_2(n+1)$$

$$W(n) = \log_2(n+1) \text{ for } n = 2^k - 1$$

$W(n) = \log_2(n+1)$ for $n = 2^k - 1$. Now let's generalize this result as follows: $W(n) = \dots$ for all n .

$2^{i-1} \leq n \leq 2^i - 1$ $W(\dots)$ is a decreasing function. $W(2^{i-1}-1) \leq W(n) \leq W(2^i-1)$

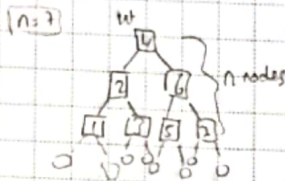
$\log_2(2^{i-1}) \leq W(n) \leq \log_2(2^i)$ $i-1 \leq W(n) \leq i$ Now I need to express it in terms of n . $2^{i-1} \leq n+1 \leq 2^i$
 $\log_2(2^{i-1}) \leq \log_2(n+1) \leq \log_2(2^i)$
 $\log_2(n+1) \leq i-1 \leq W(n) \leq i \leq \log_2(n+1)+1$ $W(n) \in (\log_2 n)$ for all n .
 In fact, this method is called interpolation.

Average Case: Assume that the probability of successful search is P , ($0 \leq P \leq 1$)

It is equally likely that x can be found in any position.

If x is not in the list, it is equally likely that it can occur in any of the $n+1$ intervals:

$x < L[1], L[1] < x < L[2], \dots, L[n-1] < x < L[n], x > L[n]$



This tree has n internal nodes and $n+1$ leaves corresponding to $n+1$ unsuccessful situations.

Probably that x can be found in any position $L[i] = \frac{P}{n}$ for $1 \leq i \leq n$ average number comparisons

If $x = L[4] \Rightarrow \frac{P}{n} \cdot 1$ just 1 comparison

If $x = L[2] = \frac{P}{n} \cdot 2$

If $x = L[6] = \frac{P}{n} \cdot 2$

$$\frac{P}{n} \cdot 1 + \frac{P}{n} \cdot 2 + \dots + \frac{P}{n} \cdot 2 = \frac{P}{n} (1 + 2 + 2 + \dots + 3 + 3 + 3 + \dots)$$

this corresponds to the internal path length of the tree $+1$

Internal Path Length: The sum of the lengths of the paths from the root to the interval nodes.

Average number of comparisons = $\frac{P}{n} \cdot (IPL(T) + n)$ for successful case

Now the unsuccessful case:

Probably that (when x is not in the list) x falls into any one of the $n+1$ intervals = $\frac{1-P}{n+1} \cdot (3+3+3+\dots)$

Leaf Path Length: The sum of the lengths of the paths from the root to the leaf nodes $\Rightarrow$ Average number of comparisons in the unsuccessful case

$$A(n) = \sum_{I \in T(n)} T(I) \cdot P(I) = \frac{P}{n} \cdot (IPL(T) + n) + \frac{1-P}{n+1} \cdot LPL(T)$$

$$= \frac{1-P}{n+1} \cdot (LPL(T) + n)$$

Definition: A 2-tree is binary tree such that every node that is not a leaf has exactly 2 children.

Definition: A binary tree T is called a full binary tree at level i if T contains the maximum number of nodes at that level, $0 \leq i \leq D(T)$

Definition: A binary tree T_n is called a full binary tree if T is full at every level i , for $0 \leq i \leq D(T)$

Definition: A 2-tree T is full at the second deepest level if all leaf nodes are in the levels $D(T)-1$ and $D(T)$. For a 2-tree T , $IPL(T) = LPL(T) - 2I$, where I is number of internal nodes.

For any binary tree T , $LPL(T) \geq 2 \cdot \lceil \log_2 L \rceil + 2(L - 2^{\lceil \log_2 L \rceil})$, where L is the number of leaf nodes.

$$A(n) = \frac{P}{n} \cdot (IPL(T) + n) + \frac{(1-P)}{n+1} \cdot LPL(T) = \frac{P}{n} (LPL(T) - n) + \frac{(1-P)}{n+1} \cdot LPL(T)$$

Since the tree of this type is a 2-tree

$$\geq \left(\frac{P}{n} + \frac{1-P}{n+1} \right) \cdot (n+1) \cdot \lceil \log_2(n+1) \rceil + 2 \cdot (n+1) - 2$$

$$\geq \left(\frac{P}{n} + \frac{1-P}{n+1} \right) \cdot (n+1) \lceil \log_2(n+1) \rceil - 2 \rightarrow n \log n \Rightarrow A(n) \in \Omega(\log n)$$

$\Rightarrow$ There are $n+1$ leaf nodes and n internal nodes.

$A(n) \leq W(n) \in O(\log n)$
Therefore, $A(n) \in O(\log n)$

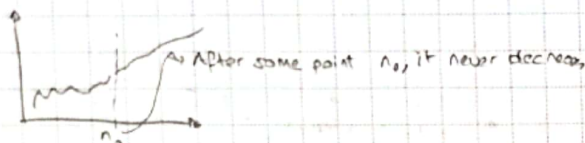
Interpolation

We know that $f(n) \in O(g(n))$ for $n = b^k$ ($k \geq 0, b > 1$)

$\Downarrow$
 $[f(b^k) \in \Theta(g(b^k)) \text{ for } n \in \mathbb{N}]$

We want to show that $f(n) \in O(g(n))$ for all $n \in \mathbb{N}$.

A function $h(n)$ is called eventually non-decreasing if there exists an integer n_0 such that $h(n) \leq h(m)$ whenever $m \geq n \geq n_0$.



For a function $h(n)$, if $h(c \cdot n) \in \Theta(h(n))$ for any positive constant c , then $h(n)$ is called Θ -variant under scaling and $\Theta(h(n))$ called a scale invariant class.

Ex) $h(n) = n^2 \in \Theta(n^2)$
 $h(c \cdot n) = c^2 \cdot n^2 \in \Theta(n^2)$
constant

Ex) $h(n) = \log n \in \Theta(\log n)$
 $h(c \cdot n) = \log(c \cdot n) = \log c + \log n \in \Theta(\log n)$

Ex) $h(n) = 2^n \in \Theta(n!)$ $\rightarrow$ Exponential functions are not Θ -variant under scaling.
 $h(c \cdot n) = 2^{c \cdot n} = (2^c)^n \notin \Theta(2^n)$
 $\downarrow$
 $2^{c \cdot n} \neq 2^c \cdot 2^n \in \Theta(2^{c \cdot n})$

Suppose that $f(b^k) \in \Theta(g(b^k))$ for $b > 1$ and $n \in \mathbb{N}$. Suppose also that $f(n)$ and $g(n)$ are eventually non-decreasing and $g(n)$ is Θ -variant under scaling. Then $f(n) \in \Theta(g(n))$ for all $n \in \mathbb{N}$.

Ex) $W(n) \in \Theta(\log n)$ for $n = 2^k - 1$
 $W(n)$ is eventually non-decreasing $\checkmark$
 $\log(n)$ is eventually non-decreasing $\checkmark$
 $\log(n)$ is Θ -variant under scaling $\checkmark$

Insertion Sort

```

procedure InsertionSort (L[1:n])
  for i = 2 to n do
    current = L[i]
    position = i - 1
    while (position > 1 and (current < L[position])) do
      L[position + 1] = L[position]
      position = position - 1
    end while
    L[position + 1] = current
  end for
end
  
```

We're trying to place the i^{th} element in the proper position in the already sorted part

the best choice
 $\uparrow$ for basic operation

Best Case: The basic operation executes once and the while loop terminates. This occurs if the input is already sorted!

$$B(n) = \sum_{i=2}^n 1 = n-1 \in \Theta(n)$$

Worst Case: For each iteration of the for loop, the basic operation is executed maximum number of times. This situation occurs if the list $L[1:n]$ is already sorted in reverse order.

$$W(n) = \sum_{i=2}^n (i-1) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \in \Theta(n^2)$$

Average Case: Let T_i = number of basic operations at step i for $1 \leq i \leq n-1$

$$T = T_1 + T_2 + \dots + T_{n-1}$$

each of the T_i 's are random variables.

Calculate $E[T_i]$, $E[T] = \sum_{j=1}^n j \cdot P(T_i = j)$ probability that there are j comparisons in the i th step.

1 comparison will occur if $x = L[i] > L[i-1]$

2 comparisons " " " " $L[i-2] < x < L[i-1]$

" " " " "

$i-1$ $L[1] < x < L[i-1]$
 $i-1$ $x < L[i]$

There are i intervals that x can fall into

$$P(T_i = j) = \begin{cases} \frac{1}{i}, & \text{if } 1 \leq j \leq i-1 \\ \frac{2}{i+1}, & \text{if } j = i \end{cases}$$

$$E[T_i] = \left(\sum_{j=1}^{i-1} j \cdot \frac{1}{i} \right) + (i-1) \cdot \frac{2}{i} \\ = \frac{(i-1) \cdot (i-2)}{2i} + \frac{2(i-1)}{i} = \frac{i^2 - 3i + 2 + 4i - 4}{2i} = \frac{i^2 + i - 2}{2i}$$

① A sorting algorithm is a stable sorting algorithm if given any 2 elements $L[i], L[j]$ where $i < j$ and $L[i] = L[j]$, the final positions i', j' of $L[i], L[j]$, respectively, satisfy $i' < j'$

② A sorting algorithm is an online sorting algorithm if the entire list is not input to the algorithm in advance but instead elements are added to the list over time. The algorithm maintains the dynamic list in sorted order. $\rightarrow$ Insertion sort $\rightarrow$ Quicksort is not online. $\rightarrow$ Merge sort is not online

Towers of Hanoi Problem

Procedure Towers(A, B, C, n)

if $n=1$ then

write ("move from A to C")

else

$T(n-1) \leftarrow$ call Towers(A, B, C, $n-1$) $\rightarrow$ move $n-1$ disk from A to C using C as auxiliary

$T(1) \leftarrow$ write ("move from A to C")

$T(n-1) \leftarrow$ call Towers(B, A, C, $n-1$) $\rightarrow$ move $n-1$ disk from B to C

$T(n)$ = complexity of towers of hanoi for an input of $n = 2T(n-1) + 1$ $T(1) = 1$

$$T(1) = 1$$

$$T(2) = 2T(1) + 1$$

$$T(3) = 2(2T(1) + 1) + 1$$

$$T(n) = 2^{n-1} \cdot T(1) + 2^{n-2} + 2^{n-3} + \dots + 2 + 1 = \sum_{i=0}^{n-1} 2^i = \frac{2^n - 1}{2 - 1} \rightarrow \sum_{k=0}^{n-1} \frac{2^k - 1}{2 - 1} \rightarrow T(n) \in \Theta(2^n)$$

General form of a Recurrence Relation

$$x(n) = c_1 \cdot x(n-1) + c_2 \cdot x(n-2) + \dots + c_k \cdot x(n-k) + f(n)$$

$$\underbrace{x(0), \dots, x(k-1)}_{\text{initial conditions}} \text{ are given } \underline{\text{or}} \quad x(n) = C \cdot x\left(\frac{n}{b}\right) + f(n) : x(n) \text{ is given}$$

④ These relations can be non-linear, homogeneous in homogeneous recurrence relations with constant coefficients.

Forward Substitution

$$x(n) = 2x(n-1) + 1 \quad ; \quad n > 1$$

$$x(1) = 1$$

$$\left. \begin{array}{l} x(1) = 1 \\ x(2) = 3 \\ x(3) = 7 \\ x(4) = 15 \end{array} \right\} \begin{array}{l} \text{There we try} \\ \text{to observe} \\ \text{a pattern} \end{array}$$

Guess; $x(n) = 2^n - 1$
 $\rightarrow$ then you have to prove that

$$x(n) = 2^n - 1 \Rightarrow x(n+1) = 2^{n+1} - 1$$

$$x(n) = 2x(n-1) + 1 \Rightarrow 2^n - 1 = 2(2^{n-1} - 1) + 1$$

$$2^n - 1 = 2^n - 2 + 1$$

$$2^n - 1 = 2^n - 1$$

Backward Substitution

$$x(n+1) = b_{n+1} \cdot x(n) + c_{n+1}, \quad n \geq 0, \quad x(0) \text{ is given}$$

$$x(n+1) = b_{n+1} \cdot \dots \cdot b_1 x(0) + b_{n+1} \cdot \dots \cdot b_2 c_1 + b_{n+1} \cdot b_3 c_2 + \dots + c_{n+1}$$

we can apply a trick to the original formula to obtain another formula. make a change of variable that leads to a new recurrence.

$$x(n) = b_1 b_2 \dots b_n g(n), \quad n \geq 1, \quad y(0) = x(0)$$

$$b_1 \dots b_{n+1} g(n) = b_{n+1} b_1 \dots b_n g(n) + c_{n+1}$$

$\rightarrow$ we make the coefficients the same, so that we can eliminate them.

$$y(n+1) = y(n) + d_{n+1}, \quad \text{where } d_{n+1} = c_{n+1}, \quad y(0) \text{ is, now this can be easily solved by backward solution}$$

$\rightarrow$ solve the recurrence

$$y(n) = y(0) + \sum_{i=1}^n d_i$$

$\rightarrow$ return to the original variable $x(n) = b_1 b_2 \dots b_n \left[y_0 + \sum_{i=1}^n d_i \right]$ convert this into closed form

Ex1 $x(n+1) = 3x(n) + n, n \geq 0, x(0) = 0$

$x(n) = 3^n \cdot y(n)$

$3^{n+1} y(n+1) = 3 \cdot 3^n \cdot y(n) + n, y(n) = 0, n \geq 0$

now recurrence $y(n+1) = y(n) + \frac{n}{3^{n+1}} \Rightarrow y(n) = y(n-1) + \frac{n-1}{3^n}$

$y(n) = \sum_{i=1}^{n-1} \frac{1}{3^{i+1}}$

$y(n) = y(n-2) + \frac{n-2}{3^{n-1}}$

$x(n) = 3^n \cdot \sum_{i=1}^{n-1} \frac{1}{3^{i+1}} \Rightarrow$

now if we cannot exactly solve this summation we can squeeze it between 2 integrals and find the Θ solution

$x(n+1) = a \cdot x(n) + b x(n-1), n \geq 1, x(0), x(1)$ are given

$\hookrightarrow$ second order recurrence relation

characteristic equation of this recurrence

$\alpha^2 = a\alpha + b$

$\rightarrow$ solve this equation to find the roots for α and then we express the solution of the recurrence in terms of these roots.

$x(n) = \alpha^n \Rightarrow$ final solution of this recurrence (similar to 2nd order differential equations)

$\alpha^{n+1} = a \cdot \alpha^n + b \alpha^{n-1}$

$\hookrightarrow$ divide all by α^{n-1}

$\alpha^2 = a\alpha + b$, roots: α^+ and α^-

roots α^+ and α^- ① if both α^+ and α^- are real and distinct then $x(n) = C_1 \alpha^+{}^n + C_2 \alpha^-{}^n$

② if $\alpha^+ = \alpha^-$ (say α) and it is real then

$x(n) = C_1 \alpha^n + C_2 n \alpha^n$

$\hookrightarrow$ we can find these constants from the initial conditions

③ if roots are complex numbers.

Ex1 recurrence for Fibonacci numbers $f(0) = 0, f(1) = 1, f(n) = f(n-1) + f(n-2)$ for $n \geq 2$

$\alpha^2 = \alpha + 1$

$\alpha^2 - \alpha - 1 = 0$ roots $\alpha^+ = \frac{1+\sqrt{5}}{2}, \alpha^- = \frac{1-\sqrt{5}}{2}$

$f(n) = C_1 (\alpha^+)^n + C_2 (\alpha^-)^n$

$f(n) = C_1 \left(\frac{1+\sqrt{5}}{2}\right)^n + C_2 \left(\frac{1-\sqrt{5}}{2}\right)^n$

$f(0) = C_1 + C_2 = 0$

$f(1) = C_1 \left(\frac{1+\sqrt{5}}{2}\right) + C_2 \left(\frac{1-\sqrt{5}}{2}\right) = 1 \rightarrow \underbrace{C_1 + C_2}_{=0} + \underbrace{\frac{\sqrt{5}}{2}(C_1 - C_2)}_{=1} = 1 \Rightarrow C_1 = \frac{1}{\sqrt{5}}, C_2 = -\frac{1}{\sqrt{5}}$

$f(n) = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2}\right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2}\right)^n$

Ex1 $x(n+1) = 2 \cdot x(n) - x(n-1), n \geq 1, x(0) = 1, x(1) = 5$

$\alpha^2 = 2\alpha - 1$

$(\alpha^2 - 1)^2 = 0$

$x(n) = C_1 \alpha^n + C_2 n \alpha^n$

$x(n) = C_1 + C_2 n$

$x(1) = C_1 + C_2 = 5$

$C_1 = 1$

$C_2 = 4$

$x(n) = 4n + 1$

Ex) One type of recurrence $\Rightarrow x(n) = a \cdot x(n-1) + f(n)$; $x(1) = b$

$\hookrightarrow$ decrease by 1 algorithms (like insertion sort, factorial calculation)

② $x(n) = a \cdot x\left(\frac{n}{b}\right) + f(n)$; $x(1) = c$; $b > 1$
 $\hookrightarrow b$ usually 2

$a = 1 \Rightarrow$ decrease by a constant algorithms (binary search)

$a > 1 \Rightarrow$ divide and conquer algorithms (ex. quick sort, merge sort)

③ ① $x(n) = a^{n-1} \cdot b + \sum_{i=2}^n a^{i-1} \cdot f(i)$ (by backward substitution)

if $a = 1$ and $f(n) = 1 \Rightarrow x(n) = b + \sum_{i=2}^n 1 = b + (n-1) \in \theta(n)$

if $a = 1$ and $f(n) = \log n \Rightarrow x(n) = b + \sum_{i=2}^n \log(i) = b + \log(n!) \in \theta(n \log n)$

② if 2 is not a power of b , we'll obtain something like $x(n) = a \cdot x\left(\left\lceil \frac{n}{b} \right\rceil\right)$.
 $\hookrightarrow$ assume that $n = b^k$

Master Theorem

Let $x(n)$ be an eventually non-decreasing function that satisfied the recurrence relation:

$$x(n) = a \cdot x\left(\frac{n}{b}\right) + f(n); \quad n = b^k \quad (k = 1, 2, \dots) \quad x(1) = c \quad \text{where } a \geq 1, b \geq 2, c > 0$$

if $f(n) \in \theta(n^d)$, where $d \geq 0$, then

$$x(n) \in \begin{cases} \theta(n^d); & \text{if } a < b^d \\ \theta(n^d \log n); & \text{if } a = b^d \text{ for all } n \\ \theta(n^{\log_b a}); & \text{if } a > b^d \end{cases}$$

Quick Sort (invented by Hoare in 1962)



procedure Quicksort($L[low:high]$)

if $high > low$

call Rearrange($L[low:high]$, position)

call Quicksort($L[low; position-1]$)

call Quicksort($L[position+1; high]$)

endif

end

procedure Rearrange($L[low:high]$, position)

right = low

left = high + 1

$x = L[low]$

while right < left

repeat right = right + 1 until $L[right] \geq x$

repeat left = left - 1 until $L[left] \leq x$

if right < left then

call interchange($L[right]$, $L[left]$)

endif

end while

position = left

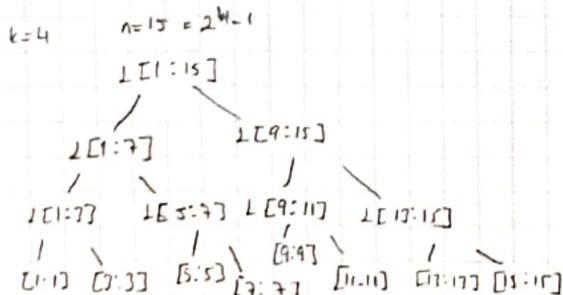
$L[low] = L[position]$

$L[position] = x$

end $\hookrightarrow$ correct position of the pivot element

Best Case: If the list is divided into equal parts, this is the best case.

Assume for simplicity that $n = 2^k - 1$ for some $k \in \mathbb{Z}^+$



In Rearrange, always high - low + 2 comparisons will be performed. The tree has k levels. In $k-1$ of these levels, there are $n+1$ comparisons.

$$B(n) = (k+1)(n+1) - (\log_2(n+1) - 1)(n+1) \in \Theta(n \log n) \text{ for } n = 2^k - 1$$

② these functions are non-decreasing and Θ -invariant, hence we can generalize this result for all n .

Worst - Case: For, quicksort, if the list is already sorted, this is the worst input.

$$1[1:15] \quad W(n) = \sum_{i \in S} i \in \Theta(n^2)$$

$$\quad \swarrow$$

$$\quad 2[2:15]$$

$$\quad \quad \swarrow$$

$$\quad \quad 3[3:15]$$

Average - Case: Assume that it is equally likely that the pivot element ($L[\text{low}]$) will be placed in any position after rearrange. (The proper position of the pivot is the important thing in this algorithm)

$T = T_1 + T_2$ $\rightarrow$ number of operations in recursive call

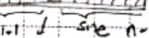
$$A(n) = E[T] = \underbrace{E[T_1]}_{\substack{\text{due to} \\ \text{high low} \\ \text{operation}}} + \underbrace{E[T_2]}_{\substack{\text{depends} \\ \text{where the} \\ \text{has been p}}}$$

If proper position of the pivot was $L[i]$, then the 2 sublists are $L[1:i]$ and $L[i:n]$

" " " " " " $L[i]$ then the 2 sublists are $L[1:i-1]$ and $L[i+1:n]$

in Variable

$$E[T_2] = \sum_{i=1}^n E[T_2 | X=i] \cdot \underbrace{P(X=i)}_{\substack{\text{Random Variable} \\ = \frac{1}{n} \text{ (due to equally likely assumption)}}}$$



$$A(n) = (n+1) + \sum_{i=1}^n [A(i-1) + A(n-i)] - \frac{1}{n}$$

Full history recurrence

$$= (n+1) + \frac{2}{n} \cdot [A(0) + A(1) + \dots + A(n-1)]$$

$\Rightarrow A(0) + A(n)$ for $i=1$
 $A(1) + A(n-2)$ for $i=2$
 $\vdots$
 $A(n-1) + A(0)$ for $i=n$

$$\begin{aligned} n \cdot A(n) &= n(n+1) + 2 \cdot [A(0) + A(1) + \dots + A(n-1)] \\ - (n-1) \cdot A(n-1) &= n(n-1) + 2 \cdot [A(0) + \dots + A(n-2)] \\ n \cdot A(n) - (n-1) \cdot A(n-1) &= 2n + 2A(n-1) \end{aligned}$$

$$\frac{1}{n(n+1)} \Rightarrow \frac{A(n)}{n+1} - \frac{A(n-1)}{n} = \frac{2}{n+1} \Rightarrow \frac{A(n)}{n+1} = \frac{A(n-1)}{n} + \frac{2}{n+1}$$

divide by $n(n+1)$

change of variable: $t(n) = \frac{A(n)}{n+1}$
 $\Rightarrow t(n) = t(n+1) + \frac{2}{n+1}$
 now its a 1st order
 recurrence
 $t(1) = \frac{A(1)}{2} = 0$ (initial condition)
 $t(0) = \frac{A(0)}{1} = 0$

$$\begin{aligned} f(0) &= 0 \\ f(1) &= 0 \\ f(2) &= f(1) + \frac{2}{2+1} \\ f(3) &= f(2) + \frac{2}{3+1} = f(1) + \frac{2}{2+1} + \frac{2}{3+1} \\ f(n) &= \sum_{i=2}^n \frac{2}{i+1} = 2 \cdot \underbrace{H(n+1) - 1}_{\text{harmonic series}} \end{aligned}$$

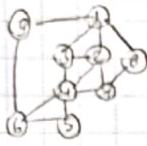
$$A(n) = 4(n), (n+1) \\ = 2(n+1) + \underbrace{4(n+1) - 3(n+1)}_{\log n \in \Theta(\log n)}$$

④ the average and best case complexities of quick sort are very close to each other.

$A(n) \approx 1.38 B(n)$ $\Rightarrow$ this has been shown in the literature

Graph Search and Traversal Algorithms

Depth-First Search (DFS)



Visit 7

Unvisited neighbours of 7: 2, 4, 8

Randomly select one such neighbour visit it

Visit 2

Unvisited neighbours of 2: 1, 3, 4

Visit 1

procedure DFS(G, v)

call Visit(v)

Mark [v] = 1

$u = v$

call Next(u, w , found)

while found or (not (Empty(stack)))

if found then

call Push(stack, u)

Mark [w] = 1

call Visit(w)

$u = w$

else

call Pop(stack, u)

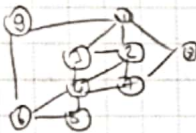
end if

call Next(u, w , found)

end while

end

Breadth-First Search (BFS)



Visit 7

Unvisited neighbours of 7: 2, 4, 8

Visit 2

Visit 4

Visit 8

Unvisited // // 2, 1, 3

Visit 1

Visit 3

Unvisited // // 4, 5, 6

Visit 5

Visit 6

Unvisited // // 8: None

Unvisited // // 1: 9

Visit 9

Unvisited // // 3: None

procedure BFS(G, v)

call visit(v)

Mark [v] = 1

Enqueue(Queue, v)

while not (Empty(Queue)) do

Dequeue(Queue, u)

for each vertex w adjacent to u do

if Mark [w] = 0 then

call Enqueue(Queue, w)

Mark [w] = 1

Visit(w)

end if

end for

end while

end

Brute Force & Exhaustive Search

Exhaustive Search is an important special case of brute force.

↳ easiest strategy to apply.

⑦ Selection sort & bubble sort are 2 well-known BF algorithms

Ex) Exponentiation

↳ Recall: Computing $a^m \bmod n$ for some large integers is a principal component of leading encryption algorithms

Selection Sort

- Scan the entire list to find its smallest element.
- Exchange it with the first element.
- Put the smallest element in its final position in its final position in the sorted list.
- Then continue with finding the second smallest etc.
- ④ If the list has size n , the list is sorted after $n-1$ passes.

Pseudocode for Selection Sort

```
Algorithm SelectionSort (A[0]..... n-1])
for i ← 0 to n-2 do
    min ← i
    for j ← i+1 to n-1 do
        if A[j] < A[min]
            min ← j
        end if
        swap A[i] and A[min]
    end for
end for
```

Analysis: $C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$

$$= \sum_{i=0}^{n-2} (n-1-i) = (n-1) \sum_{i=0}^{n-2} 1 = (n-1) \cdot \frac{(n-1)(n-2)}{2}$$

Thus selection sort is a $\Theta(n^2)$ algorithm on all input.

Bubble Sort

Compare adjacent of the list, exchange them if they are out-of-order, by doing this repeatedly we end up "bubbling up" the largest element to the post position on the list. The next pass bubbles up the the second largest until after $(n-1)$ passes, the list is sorted.

Observation: If a pass through the list makes no changes then it means that the list has been sorted and we can stop the algorithm.

Pseudocode for Bubble Sort (A[0]..... n-1])

```
for i ← 0 to n-2 do
    for j ← 0 to n-2-1 do
        if A[j+1] < A[j]
            swap A[j] and A[j+1]
        end if
    end for
end for
```

Analysis: $C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1$

$$= \sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2} \in \Theta(n^2)$$

Bubble sort is $\Theta(n^2)$ on all inputs.

Sequential Search & BF String Matching

String Matching Problem: Given a string of n characters called the text and a string of m characters called the pattern, find a substring of the text that matches the pattern.

→ We want to find i , which is the index of the leftmost character of the first matching substring in the text.

Ex) Text: NOBODY NOTICED HER

Possible application: DNA Sequence Alignment

Pattern: NOT

Pseudocode BF-String Match ($T[0, \dots, n-1], P[0, \dots, m-1]$)

```

for i ← 0 to n-m do
  j ← 0
  while j ← m and  $P[j] = T[i+j]$  do
    j ← j+1
  end while
  if j = m
    return i
  end if
end for
return -1
end

```

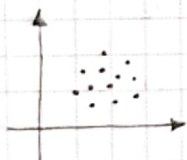
Analysis: The algo may have to make all m comparisons before shifting the pattern, and this can happen for each of the $n-m+1$ tries. Then in the worst case, the algo makes $m(n-m+1)$ character comparisons.

Complexity: $O(n, m)$

Closest Pair and Convex Hull Problems by Brute Force

These problems arise in computational geometry and operations research.

Closest Pair Problem: Finding the 2 closest points in a set of points.



① Points can represent airplanes, post offices, DB records.

② 2 closest planes are the most probable collision candidates.

③ Another app: Cluster analysis in statistics. (Similarity metric is used instead of Euclidean distance.)

④ For text and other non-numeric data, similarity metric might be the Hamming distance.

⑤ For simplicity, we consider the 2D case. 2 points $p_i(x_i, y_i)$. Distance between 2 pairs P_i and P_j :

$$= d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

BF Algorithm for Closest Pair

→ Compute the distance between each pair of points
→ Find a pair with the smallest distance.

⑥ Even today, computation of $\sqrt{}$ is costly; mainly because it involves dealing with irrational numbers. Can it be avoided? YES

Since $\sqrt{}$ is strictly increasing the basic operation will be squaring a number.

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^n 1 \rightarrow \sum_{i=1}^{n-1} (n-i) = (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2} \in \Theta(n^2)$$

Pseudocode BF-CP

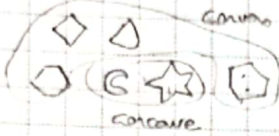
```

d ← ∞
for i ← 1 to n-1 do
  for j ← i+1 to n do
    d ← min(d,  $(x_i - x_j)^2 + (y_i - y_j)^2$ )
  end for
end for
return d

```

Closest Pair and Convex Hull Problems by Brute Force

Convex: A set of points finite or infinite in the plane is called convex if for any 2 points p and q in the set, the entire line segment with the end points at p and q belongs to the set.



Convex Hull: The convex hull of a set S of point is the smallest convex set containing set S . The smallest requirement means that the convex hull of S must be a subset of any convex set containing S .

- ① If S is convex, its convex hull is obviously S itself.
- ② If S is a set of 2 points, its convex hull is the line segment connecting them.
- ③ If S is a set of 3 points not on the same line, then its convex hull is the triangle with vertices at the given points.

Applications: Convex hulls provide convenient approximations of object shapes and data sets given. In computer animation, replacing objects by their convex hull speeds up collision detection.

- Detecting outliers by some statistical techniques.
- Finding bounding boxes.
- Visual pattern matching: Detecting licence plates.

Theorem: The convex hull of any set S of $n > 2$ points not all on the same line is a convex polygon with the vertices at some of the points of S .

↳ We call them "extreme points"

- ④ We need to find the points that will serve as the extreme points.

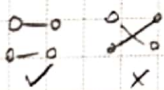
Extreme Point: of a convex set is a point of this set that is not a middle point of any line segment with end points in the set.



- ⑤ We also need to know which pair of points need to be connected to form the boundary of the convex hull.

- ⑥ We can address this issue by listing the extreme points in a clockwise or counter-clockwise order. So how we can solve this problem in BF manner?

Observation: A line segment connecting 2 points p_i and p_j of a set of n points is a part of the convex hull's boundary iff all other points of the set lie on the same side of the straight line through these points.



- ⑦ What is the time efficiency of this algorithm? For each of $\binom{n}{2} = \frac{n(n-1)}{2}$ pairs of distinct points we need to check whether all of the remaining points are on the same side. $O(n^3)$

Exhaustive Search: Combinatorial problems deal with combinatorial objects such as permutations, combinations and subsets of a given sets. Many of them are optimization problems.

Exhaustive search is simply a BF approach to combination problems. It suggest generating each and every element of the problem domain.

Traveling Salesman Problem (TSP)

To find the shortest (min-cost) tour through a given set of cities that visits each city exactly once and returns to the original city. Input is a complete graph, and the problem is to find a min-cost Hamiltonian cycle (HC).

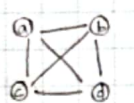


An exhaustive search approach to TSP: Generate all HC's and choose the one with minimum total edge cost.

A HC can also be defined as a sequence of $(n+1)$ adjacent vertices $(v_1, v_2, \dots, v_n, v_1)$ where the first and the last are the same.

We can generate all tours by generating all permutations of $(n-1)$ intermediate cities divided by 2
 $\rightarrow \frac{(n-1)!}{2}$ (Same as ordering a set of n integers around a ring)

$n=4 \Rightarrow \frac{3!}{2} = 3$



$(a, b, c, d, a), (a, c, b, d, a), (a, d, b, c, a)$

$\left. \begin{array}{l} b, c, d \\ b, d, c \\ c, b, d \\ c, d, b \\ d, b, c \\ d, c, b \end{array} \right\} \begin{array}{l} 3 \text{ possibilities} \\ \& \text{ permutations} \end{array}$

Knapsack Problem

n items,
 profit p_i for item i (value V_i for item i)
 weight w_i for i
 weight limit W

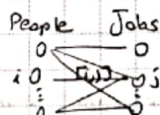
Exhaustive Search Approach: Generate or enumerate all subsets and choose the one with max. total profit adhering to the total weight limit W .
 There are 2^n subsets.

Both Traveling Salesman Problem & Knapsack are NP-Hard problems.

Assignment Problem: n problem
 n jobs
 1 person/job

$C[i, j]$ = cost of assigning person i to job j

$$i \left[\begin{array}{c} - \\ | \\ C[i, j] \\ | \\ - \end{array} \right]_{n \times n}$$



The task is to assign 1 person/job with minimum total cost.

The problem boils down to selecting 1 element in each row of the matrix so that all selected elements are in different columns and the total sum of the selected elements is the smallest possible.

We can describe all feasible solutions to the assignment problem as n -tuples $(j_1, j_2, \dots, j_n)$ in which the i^{th} component indicates the column of the selected in the i^{th} row.

For instance $(2, 3, 4, 1)$ indicates that Person-1 is assigned to Job-2

Person-2 " " " Job-3

Person-3 " " " Job-4

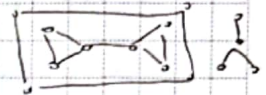
Person-4 " " " Job-1

So there are $n!$ candidates. However, Hungarian algorithm solves it in polynomial time $O(n^3)$

DFS & BFS

Systematically process all vertices and edges of a graph. Fundamental for detection of connectivity & cycle presence

- DFS** → Start at an arbitrary vertex v , mark it as "visited"
- Iteration: Proceed to an adjacent vertex (Ties are resolved arbitrarily). Continue until a dead end (a vertex with no adjacent unvisited vertices).
 - At a dead end, back up 1 edge to the vertex it came from and try to continue visiting unvisited vertices from there.
 - The algorithm eventually halts after backing up to the starting vertex.
- ④ Accompany a DFS traversal by constructing a DFS forest.



Pseudocode:

Algorithm DFS(G):

```

count ← 0
for each vertex  $v$  in  $V$  do
    if  $v$  is marked with 0,
        DFS( $v$ )
        count ← count + 1, // Mark  $v$  with count
        for each vertex  $w$  in  $V$  adjacent to  $v$  do
            if  $w$  is marked with 0 do DFS( $w$ )
    
```

How efficient is DFS?

For adjacency matrix representations: $O(V^2)$

For adjacency list representing: $O(V + E)$

④ Which data structure would be useful for DFS implementation? → Stack

- ④ Checking connectivity using DFS: When the algorithm halts, check whether all vertices have been visited.
- ④ Checking for a cycle presence using DFS: If the DFS forest (notice that DFS forest is not actually a forest) does not have back edges, the graph is acyclic.

BFS → Proceeds in a concentric manner by visiting first all vertices adjacent to a starting vertex, then others.

- ④ It is convenient to use queue as a data structure.

On each iteration

- Identify all unvisited vertices adjacent to that vertex.
- Mark them as visited.
- Add them to the queue then remove the front vertex.

Pseudocode for BFS(G)

```

0 is a mark for being "unvisited"
count ← 0
for each vertex  $v$  in  $V$  do
    if  $v$  is unmarked
        BFS( $v$ )
        BFS( $w$ )
        "visits all unvisited vertices adjacent to  $v$ "
        count ← count + 1
        initialize a queue with  $v$  and mark  $v$  with count
        while the queue is not empty do
            for each vertex  $w$  in  $V$  adjacent to the front vertex do
    
```

```

        if  $w$  is marked with 0,
            count ← count + 1
            add  $w$  to the queue
            remove the front vertex from the queue.
    
```

- ④ BFS has the same efficiency as DFS
- $O(V^2)$ for adj. matrix rep
 - $O(V + E)$ for adj. list rep.

- ④ Like DFS, BFS can be used for connectivity & acyclicity.

Main Facts About DFS & BFS

	DFS	BFS
Data Structure	Stack	Queue
Applications	conn. acyclicity	conn. acyclicity
Efficiency for adj. matrix	$O(V^2)$	$O(V^2)$
Efficiency for adj. list	$O(V+E)$	$O(V+E)$

Decrease & Conquer

→ Decrease and conquer technique is based on exploiting the relationship between a solution to a given instance of a problem and a solution to its smaller instance.

① Decrease-by-constant: The size of an instance is reduced by the same constant on each iteration of the algorithm. Typically this constant equals 1.

Ex)

① Exponentiation: Computing $a^n = a^{n-1} \cdot a = f_n \begin{cases} a \cdot f_{n-1} & \text{if } n > 0 \\ 1 & \text{if } n = 0 \end{cases}$

② Insertion Sort:

② Decrease-by-constant-factor: Suggests reducing a problem instance by the same constant factor in each iteration. In most apps, this factor equals 2.

Ex)

③ Again exponentiation:
$$a_n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ is even and positive} \\ (a^{n/2})^2 \cdot a & \text{if } n \text{ is odd and positive} \\ 1 & \text{if } n = 0. \end{cases}$$

④

③ Variable-size-decrease: The size reduction pattern varies from one iteration of the algorithm to another.

Ex)

Euclid's algorithm for computing the greatest common divisor (GCD):

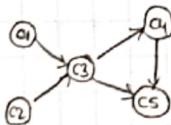
$$\gcd(m, n) = \gcd(n, m \bmod n)$$

↳ the value decreases neither by a constant nor by a constant factor.

Topological Sorting

Problem: In CS, a topological sort or topological ordering of a digraph is a linear ordering of its vertices such that for every directed edge $u \rightarrow v$ from vertex u to vertex v , u comes before v in the ordering.

Ex)



△ c_1, c_2, c_3, c_4, c_5

△ c_2, c_1, c_3, c_4, c_5

Ex) ① Critical path method in SWE (software engineering)
② Instruction scheduling in program compilation
③ Cell evaluation ordering in spreadsheet formulae
④ Resolving symbol dependencies in linker

Question: When does topological sorting not have a feasible solution?

Dis (directed acyclic graph) Answer: When the has a directed cycle.

Algorithm 1: Perform a DFS traversal & note the order in which vertices become dead ends.

C_1, C_2, C_3, C_4, C_5 C_5, C_4, C_3, C_2, C_1
 ← reverse

Algorithm 2: Direct implementation of the decrease (by-one-and-conquer) technique: Repeatedly identify the remaining graph a source vertex (a vertex with no incoming edges) note it down, and delete it along with all edges outgoing from it.

Algorithms For Generating Combinational Objects

Some type of combinatorial objects: - Permutations - Combinations - Subset of a given set.

- Ⓐ Combinatorial objects are studied in a branch of discrete mathematics called combinatorics.
- Ⓑ The number of combinatorial objects typically grows exponentially or even faster as a function of the problem size.

Our goal: Generating them not counting them (Generating permutations)

Generating Subsets

- Recall the knapsack problem
- Powerset. The set of all subsets.
- A set of size n has 2^n subsets.
- The power set of a set of size n has size 2^n

Decrease-by-1-idea: All subsets of $\{a_1, a_2, \dots, a_n\}$ can be divided into 2 graphs

- ① Sets that do not contain $\{a_n\}$
- ② Sets that contain $\{a_n\}$

Once we have all subsets of $\{a_1, a_2, \dots, a_{n-1}\}$, we can get all subsets of $\{a_1, \dots, a_n\}$ by adding $\{a_n\}$ to each subset.

	n
step 0	$\emptyset$
step 1	$\emptyset, \{a, \}$
step 2	$\emptyset, \{a, \}, \{a, a, \}, \{a, a, a, \}$
step 3	$\emptyset, \{a, \}, \{a, a, \}, \{a, a, a, \}, \{a, \}, \{a, a, \}, \{a, a, a, \}, \{a, a, a, a, \}$

- ② All 2^n bit strings $b_1, b_2, \dots, b_n$ of length n can be generated in the same way.

Bit String	000	001	010	011	...	111
Subset	$\emptyset$	$\{a_0\}$	$\{a_2\}$	$\{a_1, a_0\}$		$\{a_1, a_2, a_0\}$

④ A challenging question is whether there exists a "minimal change algorithm" for generating bit strings so that every one of them differs from its immediate predecessor by only a single bit. Answer is YES!

Ex) 000 binary reflected Gray code

001

011

Fake Gray. @ AT & T Bell Labs invented it in 1940's to minimize the effect of errors in existing digital signals.

Algorithm: Binary-Reflected-Gray-Code(n)

// Input: A positive integer n

// Output: A list of all bit strings of length n composing the Gray code

If $n=1$ make list L_1 containing bit strings 0 and 1 in this order.

Else Generate list L_1 of bit strings of size $n-1$ by calling BRGC($n-1$)

Copy list L_1 to list L_2 in reverse order

Add 0 in front of each bit string in L_1

Add 1 " " " " " " " " L_2

Append L_2 to L_1 to get list L

Return L

Decrease-by-a-Constant-Factor Algorithm

Fake-Coin Problem: Among n identical coins, one is fake. With a balance scale, we can compare any 2 sets of coins.

Divide n coins into 2 piles of $\lceil \frac{n}{2} \rceil$ coins each.

Leaving 1 coin aside if n is odd

↳ Continue dividing by 2.

→ Resulting recursion formula: $W(n) = W(\lceil \frac{n}{2} \rceil) + 1$ for $n > 1$

$W(1) = 0$

⇒ this recursion is identical to the one for the worst case number of comparisons in binary search.

④ The interesting point here is that this algorithm is not the most efficient solution. It would be more efficient to divide the coins into 3 piles $\frac{n}{3}$ coins each.

Russian Peasant Multiplication

* n & $m \Rightarrow$ positive integers

* if n is even $\Rightarrow n+m = \frac{n}{2} + 2m$

* if n is odd $\Rightarrow n+m = (\frac{n-1}{2} + 2m) + m$

④ This way, we can compute $n+m$ either recursively or directly.

Ex) n m
 20 55
 ↓
 25 170
 ↓
 6 680
 ↓
 4 1360
 ↓
 2 2720
 ↓
 1 5440

$5440 + 170 + 340 = 5950$

→ This is good for computers because multiplying by 2 and dividing by 2 is just shifting

④ Add all elements in the m column that have an odd number in the n column.

④ This method leads to very fast implementations since doubling & halving of binary numbers can be performed using binary shifts.

Computing a Median and Selection Problem

Selection Problem: Finding the k^{th} smallest element in a list of n numbers.

↳ This number is called the k^{th} order statistics. When $k = \lceil n/2 \rceil$, this number is called the median.

→ Obviously, we can find it by sorting the list and then finding the k^{th} smallest $O(n \log n)$ but this would be an overkill because we don't need the entire list, just the k^{th} smallest. Can we do better?

④ Partitioning a given list around some value p .

① Left part contains elements $< p$

② p : pivot element

③ right part contains elements $> p$.

Lomuto Partitioning

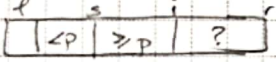
A subarray $A[l \dots r]$ ($0 \leq l \leq r \leq n-1$) composed of a continuous segments-

① If segment element known to be smaller than p

② A segment $\dots \geq p$

③ " " with elements yet to be compared to p .

→ The segments can be empty. This is always the case for the first 2 segments before the algorithm starts.

 starting with $i = l+1$, the algorithm scans the subarray $A[l \dots r]$ left to right maintaining this structure until a partition is achieved.

On each iteration

↳ Compare the first element in the unknown segment (pointed to by i) by the pivot p .

→ If $A[i] \geq p$: Increment j to expand the segment with elements $> p$ while shrinking the unprocessed segment

→ If $A[i] < p$: The segment with elements $< p$ needs to be expanded:

↳ This is done by:

① Incrementing s (the index of the first element in the first segment)

② Swapping $A[i]$ and $A[s]$

③ Incrementing i to point to the new first element of the shrunk unprocessed element

Pseudocode Lomuto Partitioning

$p \leftarrow A[l]$

$s \leftarrow l$

for $i \leftarrow l+1$ to r do:

if $A[i] < p$

$s \leftarrow s+1$

swap $(A[s], A[i])$

end if

end for

swap $(A[l], A[s])$

return i

⇒ After n of unprocessed element remain, the algorithm swaps the pivot with $A[j]$ to achieve the partition being sought.

Q) How can we take advantage of a list partition to find the k^{th} smallest element in it?



s = split position of the partition: index of the array element occupied by the pivot after partitions.

- quick select
- ① If $s = k-1$, pivot p is the k^{th} smallest element.
 - ② If $s > k-1$, k^{th} smallest element is the k^{th} smallest of the left part
 - ③ If $s < k-1$, k^{th} smallest element is the $(k-s)^{\text{th}}$ smallest of the right part

Pseudocode for Quickselect ($A[l \dots r], k$)

$s \leftarrow \text{Lomuto Partition}(A[l \dots r])$ // or another partition algorithm.

if $s = k-1$ return $A[s]$

else if $s > k-1$

 Quickselect($A[l \dots s-1], k$)

else

 Quickselect($A[s+1 \dots r], k-(s-l+1)$)

end

Q) How efficient is Quick Select?

Best Case: Partitioning on n -element array always requires $n-1$ key comparisons if we don't require more iterations, this is the best case.

$$C_{\text{best}}(n) = n-1 \in \Theta(n)$$

Worst Case: An extremely unbalanced partition (with one part empty and the other part $(n-1)$ elements)
 $\hookrightarrow$ this can happen in each of the $(n-1)$ iterations.

$$C_{\text{worst}}(n) = (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2} \in \Theta(n^2)$$

Divide & Conquer

- ① Divide the problem into several subproblems of the same type, ideally of about equal size.
- ② Solve the subproblems (typically recursively)
- ③ Combine the solutions to the subproblems to get a solution to the original problem.

$\rightarrow$ In many settings where D&C is applied, the natural BF algorithm may already be poly-time. D&C strategy may serve to reduce the running time to a lower polynomial
 $\hookrightarrow$ not always

Ex) Computing the sum of n number.

- Compute the sum of the first $\lfloor \frac{n}{2} \rfloor$ numbers
- " " " " " remaining $\lceil \frac{n}{2} \rceil$ numbers
- Add their values to get the sum in question

Ex) The natural BF approach to the closest pair problem has complexity $\Theta(n^2)$

D&C approach (we'll see) yields $O(n \log n)$

④ Typical recurrence relation for D&C:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

A function that accounts for dividing the instance of size n into instances of size $\frac{n}{b}$ & combining their solutions.

④ If $n=1$, D&C calls decrease-by-a-constant factor algorithm.

Merge Sort

→ Divide the array into 2 halves, sort each one recursively and merge them to a single sorted array.

Algorithm MergeSort($A[0, \dots, n-1]$)

if $n > 1$

copy $A[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$ to $B[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$

copy $A[\lfloor \frac{n}{2} \rfloor, \dots, n-1]$ to $C[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$

MergeSort($B[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$)

MergeSort($C[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$)

Merge(B, C, A) → Merge 2 sorted list
B & C and put the output in A

The merging of 2 sorted arrays can be done as follows:

① 2 points are initialized to point to the first element of the arrays.

② Compare the elements pointed to, copy the smaller one to the array & increment the index of the smaller element.

④ Repeat this operation until one of the 2 given arrays is exhausted. Then, the remaining elements are copied to the remaining elements are copied to the end of the new array.

Algorithm Merge($B[0, \dots, p-1], C[q, \dots, p-1], A[0, \dots, p+q-1]$)

$i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$

while $i < p$ and $j < q$

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i]$

$i \leftarrow i+1$

else

$A[k] \leftarrow C[j]$

$j \leftarrow j+1$

end

end

$k \leftarrow k+1$

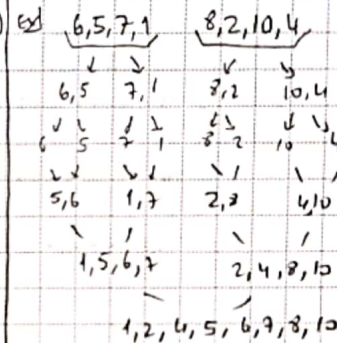
if $i = p$

copy $C[j, \dots, q-1]$ to $A[k, \dots, p+q-1]$

else

copy $B[i, \dots, p-1]$ to $A[k, \dots, p+q-1]$

end



$$C(n) = 2C\left(\frac{n}{2}\right) + C_{\text{merge}}(n)$$

At each step, exactly 1 comparison is needed in the worst case, neither of the 2 arrays become empty before the other one contains just 1 element. Hence, $C_{\text{merge}}(n) = n-1$,

and we have $C_{\text{worst}}(n) = 2 \cdot C_{\text{worst}}\left(\frac{n}{2}\right) + n-1$

$$C_{\text{worst}}(n) \in n \log n$$

What if n is odd? $T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) - \frac{n}{2} - 1$

Asymptotic bounds do not change

Multi-way Merge Sort

① A noteworthy advantage of merge sort over quicksort and its stability.

② The principal shortcoming of merge sort is the linear amount of extra storage the algorithm requires.

Quick sort

Unlike mergesort, which divides its input elements according to their position, quicksort divides them according to their value.

Partition $\Rightarrow$ A partition is an arrangement of the array's elements $\Rightarrow$ that all elements to the left some element $A[i]$ are less than $A[i]$
 " " " " right " " $A[i]$ " greater than "

$$\underbrace{A[0], \dots, A[i-1], A[i]}_{\text{all are } \leq A[i]} \quad \underbrace{A[i+1], \dots, A[n-1]}_{\text{all are } \geq A[i]}$$

→ In merge sort → The division is obvious and the entire work happens in combining their solutions

→ In quicksort → The entire work happens in the division stage, with no work required to combine the solutions to the subproblems.

Algorithm QuickSort ($A[l, \dots, r]$)

if $l < r$

$s \leftarrow \text{partition}(A[l:r], r)$ // s is a split position $\rightarrow$ As a partitioning algo, we can use for loops or another one.

Quel desort (A[1], ..., s-1]

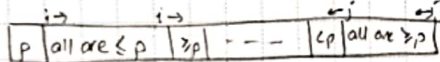
Quicksort ($A[s+1, \dots, r]$)

- For pivot, do the simplest selection: Choose $p = A[1]$

② Unlike to Quicksort algo, now scan the subarray from both ends, comparing the subarray's elements with the pivot.

f: Denotes the left-to-right scan & starts with the second element. This scan stops upon encountering the first element $\geq$ pivot

j: " " right-to-left " " " " last " " " " " " " first element
≤ pivot



④ After both scans stop, 3 situations may arise:

① If i_0 and j_0 have not crossed (i.e. $i_0 < j_0$), we simply exchange $A[i_0]$ and $A[j_0]$ & resume the scans by incrementing i_0 & decrementing j_0 .

② If i and j have crossed (i.e. $i > j$), we will have partitioned the subarray after exchanging the pivot with $A[j]$

③ If i, j, s stop while pointing to the same element (i.e. $i=j$), the value they are pointing to must be equal to p . Thus we already have the subarray partitioned with the split position $S=i=j$

Algorithm Hoare Partition ($A[1, \dots, r]$)

$$p \leftarrow A[u]$$
$$i \leftarrow l$$

j ← 1

report

repeat $i \leftarrow i+1$ while $A[i] \geq p$

repeat $j \leftarrow j - 1$ until $A[j] \leq p$

$$\text{swap}(A[i], A[j])$$

until $i \geq j$

swap ($A[i]$, $A[j]$) // and last swap when $i \geq j$

$$\text{swap}(MLJ, AC; J)$$

return j

Efficiency:

The number of comparisons made before a partition is achieved = $\begin{cases} n+1; & \text{if the scanning indices cross over} \\ n; & \text{if they coincide} \end{cases}$

→ If all splits happen in the middle of the corresponding subarrays, we have the best case.

$$C_{best}(n) = 2 \cdot C_{best}\left(\frac{n}{2}\right) + n \text{ for } n > 1, C_{best}(1) = 0$$

acc. to the Master theorem, $C_{\text{best}}(n) \in \Theta(n \log n)$

Worst-Case: In the worst case, all splits are skewed to the extreme:

- 1 subarray is empty.
- The other one has size 1 less than the input

This situation arises for all iterations.

This occurs if the input is already sorted

$$\left. \begin{array}{cccc} (n+1) & \text{comparisons} & \text{in iteration} & 1 \\ n & " & " & 2 \\ \vdots & \vdots & \vdots & \vdots \\ 3 & " & \text{find iteration} & \end{array} \right\} = \frac{(n+1)(n+2)}{2} - 3 \in O(n^2)$$

Average-Case: A partition can happen in any position s ($0 \leq s \leq n-1$) after $n+1$ comparisons are made to achieve the partitions. After the partition, left and right subarrays will have s and $(n-1-s)$ elements.

→ Assuming that the partition split can happen in each position $\leq$ with the same prob. $\frac{1}{n}$, we get the following recurrence:

$$C_{avg}(n) = \frac{1}{n} \cdot \sum [C_{avg}(s) + C_{avg}(n-1-s) + (n+1)] \text{ for } n \geq 1$$

→ $C_{avg}(n) \in \Theta(n \log n)$

Counting Inversions

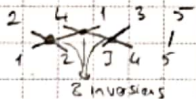
- ② 2 indices $i < j$ form an inversion if $a_i > a_j$ ("out of order"), it is called an inversion
- ③ This problem arises in the analysis of rankings

Ex) Collaborative Filtering: They try to match your preferences (for books, movies, restaurants) with those of other people out on the Internet.

④ Another application: meta-search tools $\Rightarrow$ Execute the same query on many different search engines. Then synthesize the results by comparing their rankings.

A core issue: The problem of comparing 2 rankings.

Geometric way:-



Each crossing pair of line segments corresponds to 1 pair that is in the opposite order in the 2 list.

Designing & Analyzing The Algorithm

Simplest algorithm: (Ex. Srch.) Check each pair: $O(n^2)$ time

Now we'll show a way (Div & Con) to count them in $O(n \log n)$ time.

Such an algorithm should behave without looking at each inversion individually.

- Set $m = \lfloor \frac{n}{2} \rfloor$ and divide the list into the 2 pieces $a_1, \dots, a_m$ and $a_{m+1}, \dots, a_n$
- Then count the number of inversions (a_i, a_j) , where the 2 numbers belong to different halves. we must do this part
- These inversions have a nice form: They are precisely the pairs (a_i, a_j) , where a_i is in the first half and a_j is in the second half, and $a_i > a_j$. in $O(n)$ time

- ④ we will make the algorithm recursively set the number in the 2 halves as well.

- After sorting, we have 2 sorted lists A and B , containing the first and second halves, respectively. We want to produce a single sorted list C from their union, while also counting the number of pairs (a, b) with $a \in A$, $b \in B$ and $a > b$.
- In each step, have a current pointer into each list suppose that these pointers are currently at elements.
- In each step, compare the elements a_i and b_j being pointed to in each list, remove the smaller one from the list and append it to the end of the list C .
- Every time the element a_i is appended, no new inversions are encountered since a_i is smaller than everything left in list B .
- On the other hand, if b_j is appended to list C , then it is smaller than all the remaining in A and comes after all of them. So, we increase our count of the number of inversions by the number of remaining elements in A .
↳ This way, in constant time, we have accounted for a potentially huge number of inversions.

Pseudo code:

Merge-and-Count (A, B)

- maintain a current pointer into each list, initialized to point to the front elements.
- maintain a variable count for the number of inversions, initialized to 0.
- while both lists are nonempty:
 - let a_i and b_j be the elements pointed to by the current pointers.
 - if b_j is the smaller element, then increment count by the number of elements remaining in A .
 - else advance the current pointer in the list from which the smaller element was selected.

Once the list is empty, append the remainder of the other list to the output. Return count and the merged list.

Running Time of Merge and Count:

- Each iteration of the while loop takes constant time
- In each iteration, we add some element to the output that will never be seen again.
- Thus, the number of iterations can be at most the sum of the initial lengths of A & B , so the total running time is $O(n)$

Pseudo Code for Sort-and-Count (L):

- if the list has 1 element, then there are no inversions.
- else:
 - Divide the list into 2 halves:
 - A contains the first $\lceil n/2 \rceil$
 - B " " remaining $\lceil n/2 \rceil$
 - $(r_A, A) = \text{Sort-and-Count}(A)$
 - $(r_B, B) = \text{Sort-and-Count}(B)$
 - $(r, L) = \text{Merge-and-Count}(A, B)$
- end if
- Return $r = r_A + r_B + r$ and the sorted list L

Since Merge-and-Count takes $O(n)$ time, the entire running time satisfies the recurrence:

$$T(n) \leq 2T\left(\frac{n}{2}\right) + c_n$$

$$\Downarrow$$

$$T(n) \in O(n \log n)$$

Finding the Closest Pair of Points

The set of points $P = \{p_1, \dots, p_n\}$, where p_i has coordinates (x_i, y_i) .

$d(p_i, p_j)$ = The Euclidean distance between 2 points.

Goal: To find pair of the points p_i, p_j that minimizes $d(p_i, p_j)$.

→ Assume that no 2 points have the same x -coordinate or y -coordinate.

→ Divide the area into 2 halves

 ↳ Find the closest pair in the left half

 ↳ " " " " " " " " right half

 ↳ Then use this information to get the overall solution in linear time [Combining step]

Combining Step: The distances that have not been considered by either of the recursive calls are precisely those points that occur between a point in the left half and a point in the right half.

↳ There are $O(n^2)$ such distances and we need to find the smallest one in $O(n)$ time.

On a set $P' \subseteq P$, every recursive call begins with 2 lists:

→ A list P'_x , in which all points in P' have been sorted by increasing x -coordinate.

→ A list P'_y , in which all points in P' have been sorted by increasing y -coordinate.

$Q \triangleq$ The set of points in the first $\lceil \frac{n}{2} \rceil$ positions of the list P_x (the left half)

$R \triangleq$ The set of points in the final $\lfloor \frac{n}{2} \rfloor$ positions of the list P_x (the right half)

→ By simply pass through each of P_x and P_y (in $O(n)$ time) we can create the following 4 lists:

① $Q_x \triangleq$ Points in Q sorted by increasing x -coordinate

② $Q_y \triangleq$ " " " " " " " " y - " "

③ $R_x \triangleq$ " " " " " " " " x - " "

④ $R_y \triangleq$ " " " " " " " " y - " "

For each entry (point), we record its position in the list it belongs to.

Combining the Solutions:

q_0^* and q_1^* = closest pair of points in Q

r_0^* and r_1^* = " " " " " " " " R

Let $\delta = \min \{d(q_0^*, q_1^*), d(r_0^*, r_1^*)\}$

→ Are there pairs $q \in Q$ and $r \in R$ for which $d(q, r) < \delta$?

$x^* \triangleq$ x -coordinate of the rightmost point in Q .

Lemma: If there exist $q \in Q$ and $r \in R$ for which $d(q, r) < \delta$, then each of q and r lies within a distance δ of line L_x .

Proof: Suppose that such q and r exist. We write $q = (q_x, q_y)$ and $r = (r_x, r_y)$. By definition of x^* , we know that $q_x \leq x^* \leq r_x$. Then we have: $x^* - q_x \leq r_x - q_x \leq d(q, r) < \delta$

$r_x - x^* \leq r_x - q_x \leq d(q, r) < \delta$
since $q_x \leq x^*$

So each of q and r has an x -coordinate within δ of x^* and lies within distance δ of the line L_x .

→ So, if we want to find a close q and r , we can restrict our search to the narrow band consisting only of points in P within δ of L_x .

$Q \subset R \iff$ if and only if



A restatement of the previous lemma:

There exist $q \in Q$ and $r \in R$ for which $d(q, r) < \delta$ iff there exist $s, s' \in P$ for which $d(s, s') < \delta$.

So if we want to find a close q and r , we can restrict our search to the narrow band consisting only of points in P within δ of L .

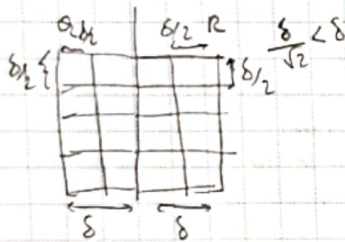
Let $S \subseteq P$ denote this set. (a set of points in P within δ of L)

S_y : Set of points in S sorted by increasing y -coordinate.

By a single pass through the list P_y , we can construct S_y in $O(n)$ time.

Lemma: If $s, s' \in S$ we have the property that $d(s, s') < \delta$, then s and s' are within 15 positions of each other in the sorted list S_y .

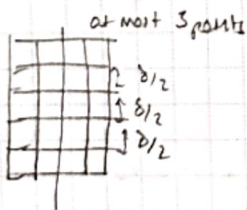
Proof: Consider the subset Z of the plane consisting of all points within distance δ of L . We partition Z into boxes as in here.



We first prove that each box contains at most 1 point of S . Suppose that 2 points of S lie in the same box. Since all points in this box lie on the same side of the line L , these 2 points either both belong to Q or both belong to R . However, any 2 points in the same box are within distance $\frac{\delta\sqrt{2}}{2} < \delta$, which contradicts our definition of δ as the minimum distance between any pair of points in Q or in R . Thus, each box contains at most 1 point of S .

Now we suppose that $s, s' \in S$ have the property that $d(s, s') < \delta$ and that they are at least 16 positions apart in S_y . Assume wlog that s has the smaller y -coordinate. Then, since there can be at most 1 point per box, there are at least 3 rows of Z , lying between s and s' .

However, any 2 points in Z separated by at least 3 rows must be a distance of at least $\frac{3\delta}{2}$ apart, a contradiction.



at most 3 points
10 boxes at least
 $\Downarrow$
is sufficient to be
put in the 2 rows

So, I have to have
at least 3 rows in between
 s and s'

Pseudocode Closest Pair (P)

Construct P_x and P_y [$O(n \log n)$ time]

$(P_x', P_y') = \text{Closest-Pair-Rec}(P_x, P_y)$

Pseudocode Closest-Pair-Rec (P_x, P_y)

if $|P| \leq 3$ then

And the closest pair by measuring all pairwise distances

end if

Construct Q_x, Q_y, R_x, R_y [$O(n)$ time]

$(q_0^x, q_1^x) = \text{Closest-Pair-Rec}(Q_x, Q_y)$

$(r_0^x, r_1^x) = \text{Closest-Pair-Rec}(R_x, R_y)$

Thm: The running time of the algorithm is $O(n \log n)$

Proof: The initial sorting takes $O(n \log n)$

The rest satisfies the DSC recurrence, hence it has running time $O(n \log n)$

$\rightarrow b = \min(d(q_0^x, q_1^x), d(r_0^x, r_1^x))$

$x^b = \text{maximum } x \text{ coordinate of a point in set } Q$

$L = \{(x, y) : x \leq x^b\}$

S = points in P within distance δ of L

Construct S_y [$O(n)$ time]

For each point $s \in S_y$, compute distance from s to each of the next 15 points in S_y

Let s, s' be the pair achieving the minimum of these distances [$O(1)$ time]

if $d(s, s') < \delta$ then

return (s, s')

else if $d(q_0^x, q_1^x) < d(r_0^x, r_1^x)$ then

return (q_0^x, q_1^x)

else

return (r_0^x, r_1^x)

end if

DYNAMIC PROGRAMMING

"Programming" starts for "planning". Dynamic Programming is a technique for solving problems with overlapping subproblems. Typically, the subproblems arise from a recurrence relating a given problem's solution to its smaller subproblems. Rather than solving overlapping subproblems again and again, solve each of the smaller subproblems only once & record the results in a table from which a solution to the original problem can then be obtained.

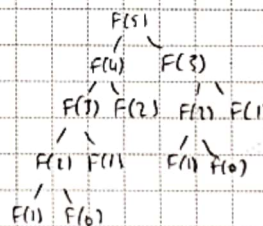
Fibonacci Number

$\hookrightarrow$

$F(0) = 0$

$F(1) = 1$

$F(n) = F(n-1) + F(n-2)$ for $n \geq 1$



0	1	1	2	3	5
---	---	---	---	---	---

$F(0) \ F(1) \ F(2) \ F(3) \ F(4) \ F(5)$

If we use this recurrence directly to compute the n^{th} Fibonacci number, as we see on this figure, we would have to recompute the same values of this function many times.

Instead, we can simply fill elements of a 1-D array by using the initial conditions & fill the rest.

Algorithm Fibonacci (n)

$F[0] \leftarrow 0$

$F[1] \leftarrow 1$

for $i \leftarrow 1$ to n do

$F[i] \leftarrow F[i-1] + F[i-2]$

end for

return $F[n]$

Dynamic Programming leads to space-time trade-off since it uses more space to save from time.

① One variation of DP approach seeks to avoid solving unnecessary subproblems. This technique exploits the so-called memory functions and can be considered as a top-down variation of DP.

Crucial Step: Deriving the recurrence relation.

Principle of Optimality: An optimal solution to any instance of an optimization problem is composed of optimal solutions to its subinstances. For instance, this principle does not exist in longest-path but it exists in shortest-path.

Coin Row Problem: There is a row of coins whose values are some positive integers $c_1, c_2, \dots, c_n$ not necessarily distinct. The goal is to pick up the maximum amount of money subject to the constraint that no 2 coins adjacent in the row can be picked up. Let $F(n)$ = maximum amount that can be picked up from a row of n coins.

Partition the allowed coin selections into 2 groups:

- ① Those that include the last coin
- ② Those that do not include the last coin

The largest amount we can get from ①: $c_n + F(n-2)$
 The largest amount we can get from ②: $F(n-1)$

Hence, we have the following recurrences:
 $F(n) = \max \{ c_n + F(n-2), F(n-1) \}$ for $n \geq 2$
 $F(1) = c_1$
 $F(0) = 0$

Algorithm Coin Row ($C[1..n]$)

```

P[0] ← 0
P[1] ← C[1]
for i ← 2 to n do
    P[i] ← max (C[i] + P[i-2], P[i-1])
end for
return P[n]
    
```

Example: Given coin row of denominations:

5, 1, 2, 10, 6, 2

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15	15	17

$(2, 5)$ → $(2+5, 15)$
 $(6, 2)$ → $(6+2, 15)$
 $(10, 6)$ → $(10+6, 17)$

Change Making Problem

→ DP algorithm for the general case, assuming a variability of unlimited quantities of coins for each of the m denominations $d_1, d_2, \dots, d_m$ where $d_i \geq 1$. Let $F(n)$ = minimum num of coins whose values add up to n . ($= \text{OPT}(n)$), $F(0) = 0$

Ⓐ The amount n can only be obtained by adding 1 coin of denomination d_j to the amount $n-d_j$ for $j=1, 2, \dots, m$ such that $n \geq d_j$. Hence, we can consider all such denominations and select the one minimizing $F(n-d_j) + 1$

Let $F(n)$ = minimum of coins whose value add up to n .
 $F(0) = 0$

Ⓑ The amount n can only be obtained by adding 1 coin of denomination d_j to the amount $n-d_j$ for $j=1, 2, 3, \dots, m$ such that $n \geq d_j$. Hence, we can consider all such denominations and select the one minimizing $F(n-d_j) + 1$. So, we have the following recurrence:

$F(n) = \min \{ F(n-d_j) \} + 1$ for $n > 0$; $n \geq d_j$. Again, we can fill a 1-row table. However, this time, computing a table entry requires finding the minimum up to n numbers.

Algorithm Change Making ($D[1..m], n$)

```

F[0] ← 0
for i ← 1 to n do
    temp ← ∞
    j ← 1
    while j ≤ m and D[j] ≤ i do
        temp ← min(F[i - D[j]], temp)
        j ← j + 1
    end while
    F[i] ← temp + 1
return F[n]

```

Application of the algorithm for $n=0$ and denominations 1, 3, 4:

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	2

$F(1) = \min(F[1-1]) + 1 = 1$
 $F(2) = \min(F[2-1], F[2-3]) + 1 = 1 + 1 = 2$
 $F(3) = \min(F[3-1], F[3-3]) + 1 = \min(2, 0) + 1 = 1$

• Space efficiency of the algorithm: $O(n)$

• Time " " " " : $O(n \cdot m)$. There are m denominations and n entries at most m comparisons are made for each entry.

Knapsack Problem

Given n items with known weights $w_1, w_2, \dots, w_n$ and values $v_1, v_2, \dots, v_n$ and a knapsack capacity. For a DP algorithm, first derive the recurrence relation. Consider an instance with the first i items $1 \leq i \leq n$.

Example	Item	weight	value	capacity = $W=5$	with weights $w_1, w_2, \dots, w_n$ and values $v_1, v_2, \dots, v_n$ and knapsack capacity $j = [1 \leq j \leq W]$	
	1	2	\$12			0 1 2 3 4 5
	2	1	\$10			0 0 0 0 0 0
	3	3	\$20			1 0 0 12 12 12 12
	4	2	\$15			2 0 10 12 21 22 22
						3 0 10 12 22 30 32
						4 0 10 15 25 30 37

Let $F[i, j]$ = The value of an optimum solution to this instance

The value of the most valuable subset of the first i items that fit into the knapsack with capacity j .

We can divide all feasible solutions to this instance into 2 categories.

- ① The subsets that do not include the i^{th} item : $F[i-1, j]$
- ② The " " " " do include the i^{th} item : $F[i-1, j-w_i] + v_i$

Of course if the i^{th} item does not fit into the knapsack, then $F(i, j) = F(i-1, j)$ (case 1)

Hence, we have the following recurrence:

$$F(i, j) = \begin{cases} \max \{ F(i-1, j), v_i + F(i-1, j-w_i) \} & \text{if } j-w_i \geq 0 \\ F(i-1, j) & \text{if } j-w_i < 0 \end{cases}$$

Initial conditions: $F(0, j) = 0$ for $j \geq 0$ and $F(i, 0) = 0$ for $i \geq 0$

• Our goal is to find $F(n, w)$. Make a 2-D table:

0	0	...	$j-w_i$	...	j	...	w
i-1							
i							
n							$F(n, w)$

Space efficiency: $O(n \cdot w)$

Time efficiency: $O(n \cdot w)$

Greedy Algorithms

→ An algorithm is greedy, if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criteria.

Ex) Prim's Algorithm: Start with a root node & greedily grow outward.
Kruskal's Algorithm: Successively insert edges in order of increasing cost.

Interval Scheduling: The Greedy Algorithm Stays Ahead

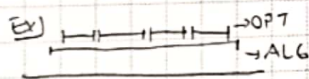
$s(i)$ = starting time of request i ,
 $f(i)$ = finish " " " " i ,

④ We say that a subset of the requests is compatible if no 2 of them overlap in time, and our goal is to accept as large a compatible subset as possible.

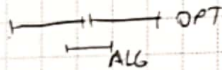
Basic Idea: Use a simple rule to select the first request i_1 .
Reject all requests that are incompatible with i_1 .
Then select the next request i_2 and again reject the ones incompatible with i_1 .
Continue until we run out of requests.

④ Which simple rule should we use?

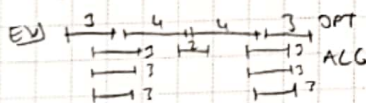
Option 1: Select the available request that starts earliest (the one with minimum $s(i)$).
↳ Does not always yield an optimum solution.

Ex) 

Option 2: Select the request that requires the smallest interval of time (with minimum $f(i) - s(i)$)



Option 3: For each request, count the number of other requests that are not compatible and accept the request with fewest number of conflicts.

Ex) 

Optimum Option: Accept the request that finishes first.